Ders 17 Çalışma Özeti

Ders 17 Çalışma Özeti

Genel Konular

- Dersi dersin hocası Dr. Ahmet Elbir anlatıyor. Konular: çok iş parçacıklı (multi-thread) programlama, POSIX thread (`pthread`) kütüphanesi ve `void*` ile genel (generic) fonksiyonlar.
- Önceki dersin konusu: çok dosyalı programların derlenmesi, `make` ve `makefile`.
- Tek akışlı ve çok akışlı programlar
  - Şimdiye kadarki programlar tek akışlıdır: `main`'den başlar, bir fonksiyona gidilir, o bitince geri dönülür. Herhangi bir anda yalnızca bir fonksiyon çalışır.
  - Çok iş parçacıklı yapıda program iş parçacıklarına (thread) bölünür ve bazı fonksiyonlar aynı anda çalışabilir. Buna görev paralelliği (task parallelism) denir.
  - Tek işlemcide gerçek eşzamanlılık yoktur. İşlemci iş parçacıkları arasında çok hızlı geçiş yapar (zaman paylaşımı), gözle fark edilmez. Hoca buna "sanal paralellik" diyor. Birden fazla işlemci varsa iş parçacıkları gerçekten aynı anda çalışabilir: gerçek paralellik.
  - Örnekler: Word'de yazarken yazım denetiminin aynı anda altını çizmesi, tarayıcıda bir sekmede müzik çalarken diğerinde e-posta okumak, Zoom'da konuşma, ekran paylaşımı ve gürültü giderme.
  - Ayrıntılar (süreç, iş parçacığı, kısıtlar) ileride İşletim Sistemleri ve Bilgisayar Organizasyonu derslerinde görülecek.
- Senkron ve asenkron
  - Senkron: işler belirli bir sırayla ve denetimle eşleştirilir. Örnek: telsiz (biri konuşur, bitirir, diğeri dinler), e-posta senkronizasyonu.
  - Asenkron: işler birbirini beklemeden yürür. Örnek: hoca konuşurken öğrencilerin aynı anda dinlemesi. Çok iş parçacıklı çalışma asenkron yürütmedir. Hangi iş parçacığının ne zaman ne kadar çalışacağı önceden bilinemez.
- Süreç (process) ve iş parçacığı (thread)
  - Süreç, çalışmakta olan programdır. Bir uygulama birden çok süreçle de yapılabilir, ama her sürecin kendi bellek bölgesi vardır. Bir süreç diğerinin belleğine erişemez, bu yüzden süreçler arası haberleşme zahmetlidir.
  - Bu nedenle bir süreç içinde birden çok iş parçacığı oluşturmak tercih edilir.
  - Zorlukları: iş parçacığı oluşturmak işletim sistemine bağlıdır (Windows'ta yazılan Linux'ta çalışmayabilir). İş parçacıkları arası haberleşme ve eşzamanlama gerekebilir. Örnek: üretici-tüketici problemi (otobüs bileti satış sitesinde firma bilet eklerken müşterilerin bilet alması).
- Bağlam değiştirme (context switching)
  - Tek işlemcide P1 ve P2 süreçleri sırayla çalışır: P1 bir süre çalışır, durdurulur, P2 çalışır, sonra P1 kaldığı yerden devam eder.
  - Bir sürecin içeriğinin (bağlamının) saklanıp diğerinin yüklenmesine context switch, bunun için geçen süreye context switching time denir. Hocanın benzetmesi: tek kasalı markette iki müşteri.
- POSIX thread (`pthread`)
  - `#include <pthread.h>`. Fonksiyon adları alt çizgiyle ayrılır (`pthread_create`, `pthread_join`). Bu bir POSIX standardıdır.
  - Her iş parçacığının benzersiz bir kimliği (thread ID) vardır ve işlemler bu kimlik üzerinden yapılır.
  - `int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void`

*(*start_routine)(void *), void *arg);
    - thread: kimliğin yazılacağı yer. İşletim sistemi kimliği atayacağı için adres (&tid) verilir.
    - attr: iş parçacığı özellikleri. Varsayılan için NULL. Özellikler konusu çok geniştir, bu derste işlenmiyor.
    - start_routine: iş parçacığının çalıştıracağı fonksiyon. void* alıp void* döndürmek zorundadır. Bu, fonksiyonun her türden parametre alıp her türden değer döndürebilmesini sağlar.
    - arg: fonksiyona gönderilecek argüman (void*). Yoksa NULL. Bir tam sayı gönderilecekse void*'a dönüştürülür.
    - Başarılıysa 0, başarısızsa bir hata numarası döndürür.
  - Örnek 1: main ve bir iş parçacığı fonksiyonu ikisi de sonsuz döngüde ekrana yazıyor. "I am main" ve "I am thread function" satırlarının hangi sırayla görüneceği önceden bilinemez. Hoca sleep ekleyerek akışı yavaşlattı.
  - Derleme: gcc program.c -o program -lpthread. Dev-C++ bunu kendisi ekliyor.
  - Örnek 2: iki iş parçacığı aynı doSomeThing fonksiyonunu çalıştırıyor. pthread_self() ile kendi kimliğini, pthread_equal(id, tid[0]) ile hangisi olduğunu anlıyor. Arada boş bir for döngüsüyle bekleme yapılıyor.
  - Değer döndürme: pthread_exit(&ret1); ile iş parçacığı sonlanırken bir adres döndürür. ret1 örnekte global değişkendir.
  - pthread_join(tid[0], (void**)&ptr[0]);: main, iş parçacığının bitmesini bekler ve döndürdüğü değeri alır. Örnekte ilk iş parçacığından 100, ikinciden 200 döner.
  - main önce biterse program kapanır ve diğer iş parçacıkları işlerini yarıda bırakır. Bu yüzden iş parçacıklarının main'e "katılması" (join) gerekir.
  - join yalnızca main'in iş parçacıklarını hangi sırayla beklediğini belirler. İş parçacıklarının bitiş sırasını ve çalışma sırasında hangi satırın önce çalışacağını belirlemez (düzeltme için "Ders sonrası notlar"a bakın).
  - Örnek 3: 5 iş parçacığı bir döngüde oluşturuluyor ve her birine sıra numarası (void*)t olarak gönderiliyor. Fonksiyon içinde (long) ile geri çevriliyor. "Hello world" mesajlarının sırası her çalıştırmada farklı olabilir.
  - İş parçacıkları aynı fonksiyonu çalıştırmak zorunda değildir. Biri f1, diğeri f2 olabilir. İkisi de void* döndürmelidir.
- Genel (generic) fonksiyonlar (Stanford CS107 slaytları üzerinden)
  - Sorun: swapInt, swapShort, swapFloat, swapPerson gibi her tür için ayrı fonksiyon yazmak yönetilemez hâle gelir.
  - Fikir: her veri sonuçta baytlardan oluşur. void* ile adresleri alıp verinin bayt sayısını ayrıca vererek her tür için çalışan tek bir fonksiyon yazılabilir. C'de çalışma anında tür bilgisi alınamaz (Java veya C#'taki gibi), ama sizeof ile boyut bilinir.
  - Genel swap:
    - void swap(void *data1ptr, void *data2ptr, size_t nbytes) { char temp[nbytes]; memcpy(temp, data1ptr, nbytes); memcpy(data1ptr, data2ptr, nbytes); memcpy(data2ptr, temp, nbytes); }
    - void* dereferans edilemediği için atama yerine memcpy kullanılır. Çağrı: swap(&x, &y, sizeof(x));. int, short, char veya bir yapı için aynı fonksiyon çalışır.
    - Hoca: temp dizisi yerine normalde malloc ile nbytes kadar yer açılır.
  - Dizinin ilk ve son elemanını değiştirme (swap_ends):
    - void swap_ends(void *arr, size_t nelems, size_t elem_bytes) { swap(arr, (char*)arr + (nelems - 1) * elem_bytes, elem_bytes); }
    - void* üzerinde adres aritmetiği yapılamadığı için dizi char*'a çevrilir ve eleman boyutuyla çarpılır.
    - Çağrı: int nums[] = {5, 2, 3, 4, 1}; swap_ends(nums, sizeof(nums) / sizeof(nums[0]), sizeof(nums[0]));.
    - Dizinin eleman boyutu fonksiyon içinde sizeof ile bulunamaz, çünkü void* ile tür bilgisi kaybolur. Bu yüzden ayrı bir parametre olarak alınır.
  - Genel kabarcık sıralaması (bubble sort):
    - Dizi void*, eleman sayısı ve eleman boyutu alınır. Karşılaştırma için bir fonksiyon işaretçisi (artan/azalan, ya da yapılarda yaş, kilo gibi bir alan) verilir. Yer değiştirme için genel swap kullanılır.
    - Bir tur boyunca hiç yer değiştirme yapılmazsa dizi sıralıdır, algoritma erken bitirilebilir. Bunun için bir bayrak (swapped) değişkeni kullanılır. Örnek: 2, 5, 4, 6, 7

dizisi bir turda sıralanır, ikinci turda hiç değişiklik olmaz ve durulur.
  - Derste gösterilen `qsort` da bu şekilde geneldir. Diğer uygulamalar: dizi döndürme (rotation), genetik algoritmada çaprazlama.
  - `void*` güçlüdür ama tehlikelidir: tür denetimi yoktur.
- Sınav vurgusu
  - Genel bubble sort yazdırılmayacak. Ama işaretçilerin birbirine dönüştürülmesi ve `sizeof` ile kullanımıyla ilgili ekran çıktısı soruları sorulabilir.

Hocanın Özellikle Vurguladığı Kısımlar

- İş parçacıklarının çalışma sırası önceden bilinemez.
- `main` bitmeden önce iş parçacıklarını `pthread_join` ile bekleyin.
- İş parçacığı fonksiyonu `void*` alıp `void*` döndürür.
- `void*` dereferans edilemez ve üzerinde aritmetik yapılamaz. `char*`'a çevirip eleman boyutuyla çarpın, kopyalama için `memcpy` kullanın.
- Final sorularında `void*`, dönüştürme ve `sizeof` ile ilgili ekran çıktısı soruları olabilir.

Kısa Tekrar Notları

- `pthread_create(&tid, NULL, fonk, arg)` → başarıda 0.
- `void* fonk(void* arg)`
- `pthread_join(tid, &donus)`, `pthread_exit(deger)`, `pthread_self()`, `pthread_equal(a, b)`
- Derleme: `gcc p.c -o p -lpthread`
- Genel swap: `memcpy` ile üç kopyalama, boyut parametresi.
- `i`. eleman adresi: `(char*)arr + i * elem_bytes`

Detaylı Açıklamalar

Genel kabarcık sıralaması (derste yazılmadı, test edildi)

```c
void bubble(void *arr, size_t n, size_t eb, int (*cmp)(const void *, const void *)) {
    int swapped = 1;
    size_t i;
    while (swapped) {
        swapped = 0;
        for (i = 1; i < n; i++) {
            void *p = (char *)arr + (i - 1) * eb;
            void *c = (char *)arr + i * eb;
            if (cmp(p, c) > 0) {
                swap(p, c, eb);
                swapped = 1;
            }
        }
    }
}
```

- `int` dizisi için `cmp`: `return *(const int*)a - *(const int*)b;`. Bir kişi yapısını yaşa göre sıralamak için: `return ((const P*)a)->yas - ((const P*)b)->yas;`. İkisi de aynı `bubble` fonksiyonuyla çalıştı.

Ders sonrası notlar

- **Kodların kontrolü:** Genel `swap`, `swap_ends`, genel kabarcık sıralaması ve slayttaki `pthread_join` örneği gcc ile (`-pthread`) derlenip denendi. `swap_ends` ile `{5, 2, 3, 4, 1}` dizisi `{1, 2, 3, 4, 5}` oldu. İki iş parçacığından 100 ve 200 değerleri alındı. Beş iş parçacıklı "Hello" örneğinde mesajlar bir denemede 1, 2, 0, 4, 3 sırasıyla geldi. Sıranın önceden bilinemeyeceğini doğrular.

- **`join` ve bitiş sırası:** Derste `join`'in iş parçacıklarının "sırayla tamamlanmasını" sağladığı söylendi (00:50:01-00:50:14). Doğrusu: `join` yalnız `main`'in bekleme sırasını belirler. Denemede 0. iş parçacığı 2 saniye uyurken 1. iş parçacığı önce bitti, `main` yine önce 0.'nın, sonra 1.'nin `join`'inden döndü (gcc ile denendi).
- **`char`'ın boyutu:** Derste bir an "karakter olsa 8 bayt" dendi. `char` her zaman 1 bayttır. 64 bit sistemde 8 bayt olan işaretçilerdir (`void*`, `char*`).
- **`size_t`:** Derste "integer türünde" dendi. Daha kesin olarak `size_t` işaretsiz (unsigned) bir tam sayı türüdür, boyut ve sayaç tutmak için kullanılır. `printf` ile `%zu` biçimiyle yazdırılır.
- **Thread kimliği:** Derste kimliğin genellikle pozitif bir tam sayı olduğu söylendi. POSIX'te `pthread_t` opak (iç yapısı tanımlanmamış) bir türdür. Bu yüzden iki kimlik `==` ile değil `pthread_equal` ile karşılaştırılır. Linux'ta pratikte bir `unsigned long`'dur.
- **`pthread_exit` ile döndürülen adres:** Örnekte `ret1` ve `ret2` global olduğu için adresleri iş parçacığı bittikten sonra da geçerlidir. Bir iş parçacığı kendi yerel değişkeninin adresini döndürmemelidir, çünkü iş parçacığı bitince o değişken yok olur. Yerine global değişken veya `malloc` ile ayrılmış bellek kullanılır.
- **`main`'de `pthread_exit(NULL)`:** Örnek 3'te `main` `return` yerine `pthread_exit(NULL)` ile bitiyor. `main`'den `return` edilirse bütün süreç kapanır. `pthread_exit` ise yalnızca ana iş parçacığını bitirir, diğerleri çalışmaya devam eder ve süreç hepsi bitince kapanır. Yine de sonucu beklemek için `pthread_join` daha açık bir yoldur.
- **`-lpthread` ve `-pthread`:** Derste verilen `-lpthread` kütüphaneyi bağlar. Günümüzde gcc için önerilen bayrak, hem derleme hem bağlama ayarlarını yapan `-pthread`'dir.
- **Değişken uzunluklu dizi:** Genel swap'teki `char temp[nbytes];` C99'un değişken uzunluklu dizi (VLA) özelliğidir. Büyük `nbytes` değerlerinde stack taşabilir. Hocanın dediği gibi `malloc` kullanmak daha güvenlidir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.